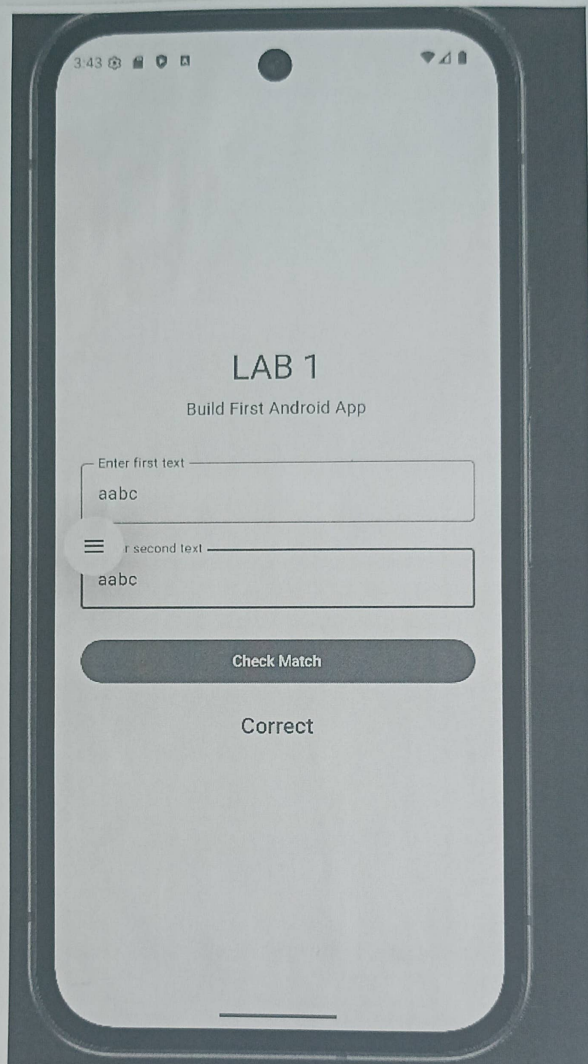




Kotlin Basics

Aim :

To understand the basic syntax and features of the kotlin programming language.

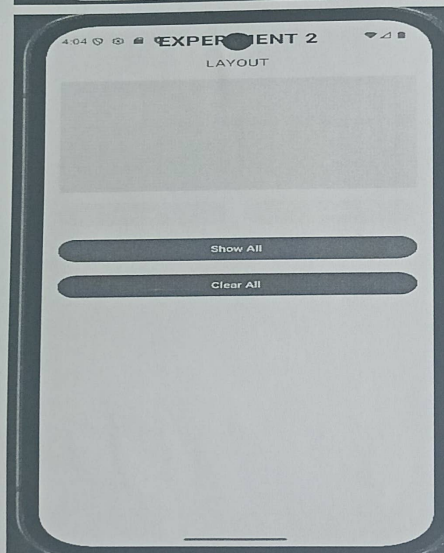
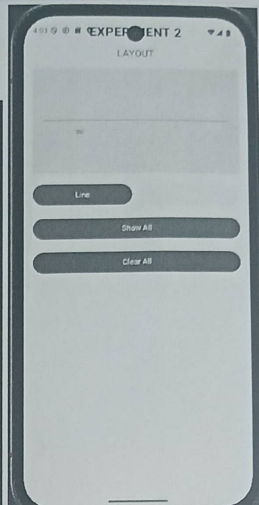
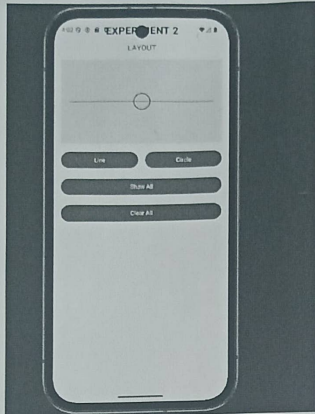
Algorithm :

- * Start the android application.
- * Create a new android project in android studio using kotlin and jetpack compose.
- * Import the required packages and libraries.
- * Run the application in an Android emulator.
- * Verify the output.

Result :

The basic kotlin program was implemented successfully.

br



Functions

Aim :

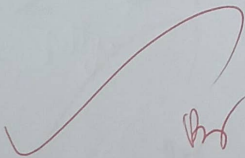
To study the implement function in Kotlin for performing specific tasks.

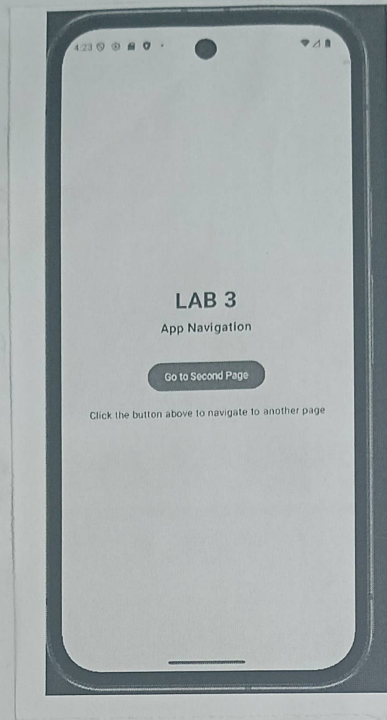
Algorithm :

- * start the android application.
- * create a new android project in android studio.
- * Import the required packages
- * Run the application in an android emulator.
- * Verify the output.

Result :

Functions were created and executed successfully.





Aim :

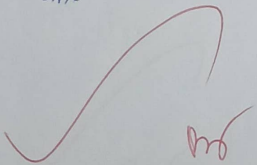
To understand object-oriented programming concept using classes and objects in Kotlin.

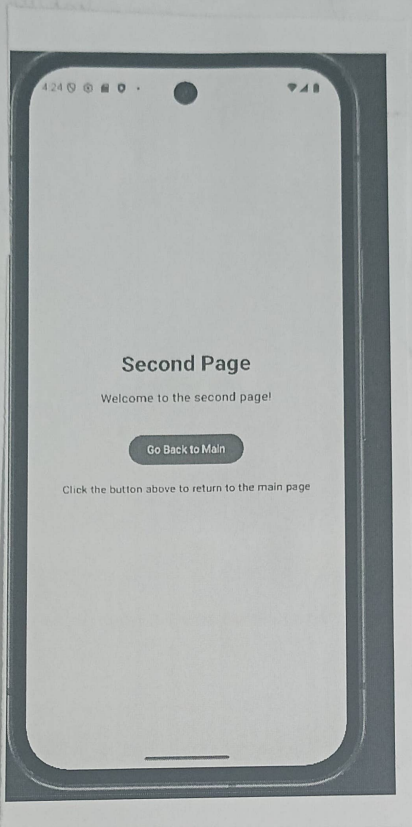
Algorithm:

- * start the Android application
- * create a new Android project in Android Studio.
- * Import the required packages
- * Run the application in android emulator.
- * Verify the output.

Result :

Classes and objects were created successfully and the concepts of object oriented programming





Aim :

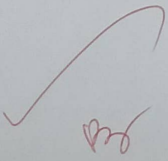
To create and run a simple android application using Android studio and kotlin.

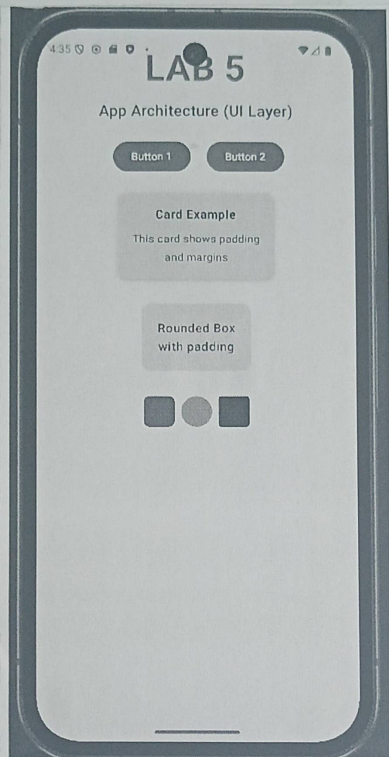
Algorithm :

- * Start the Android application.
- * create a new android project in android studio using kotlin.
- * Import the required package.
- * Run the application in Android emulator.
- * verify the output.

Result :

A basic android application was created and executed in the emulator successfully.





Layouts

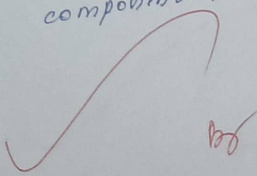
Aim : To design and implement different layout in an android application using Jetpack compose.

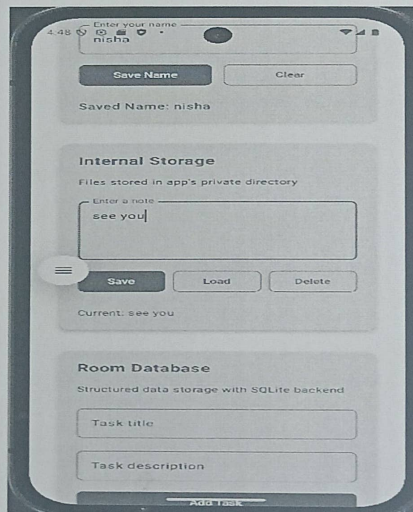
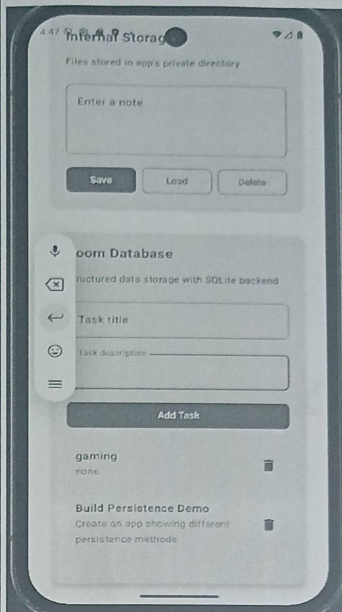
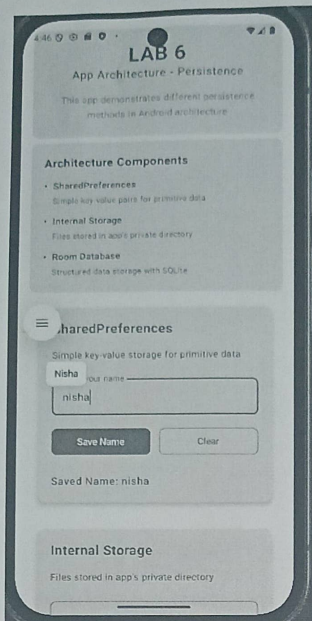
Algorithm :

- * Start the android application.
- * Create a new Android project in android studio using kotlin.
- * Import the required packages.
- * Run the application in an android emulator.

Result :

Layouts were successfully designed and UI components.





App Navigation

Aim :

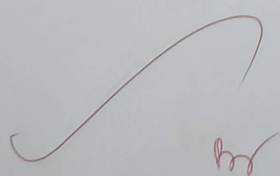
To implement navigation between different screens in an android application.

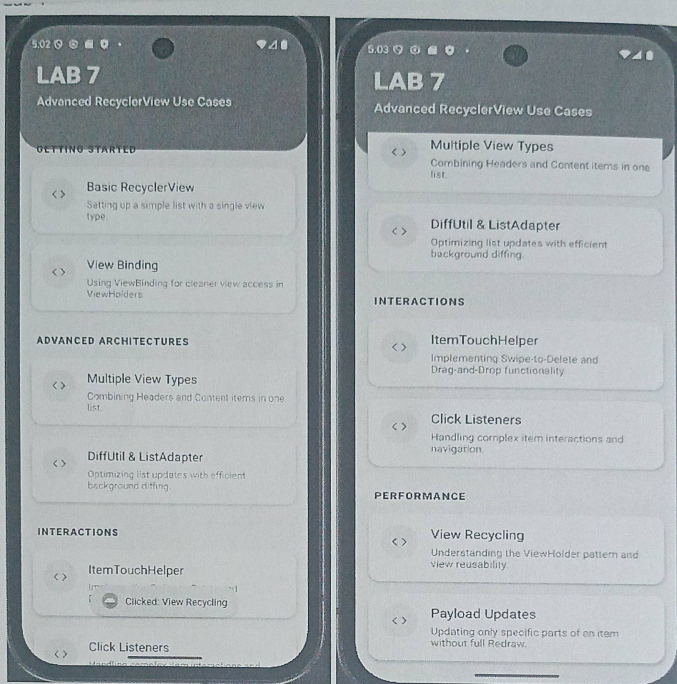
Algorithm :

- * Start the android application.
- * Create a new android project in android studio.
- * Import the required packages
- * Run the application in an android emulator.
- * Verify the output.

Result :

Navigation between multiple screens was implemented successfully.





Activity and fragment lifecycle

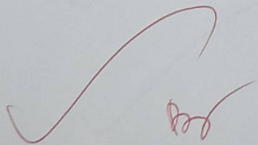
Aim : To study the lifecycle methods of activity and fragment in android.

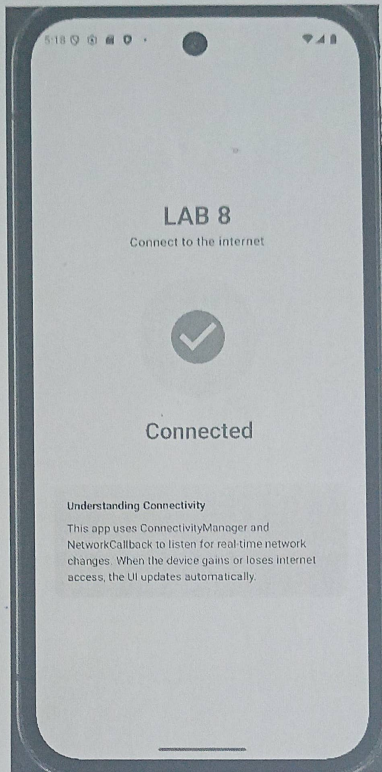
Algorithm :

- * start the Android application.
- * create a new Android project in Android studio.
- * Import the required package.
- * Run the application in an android emulator.
- * Verify the output.

Result :

The lifecycle behaviour of activity and fragment was observed and successfully.





LAB 8

Connect to the internet



Connected

Understanding Connectivity

This app uses `ConnectivityManager` and `NetworkCallback` to listen for real-time network changes. When the device gains or loses internet access, the UI updates automatically.

Aim: To understand and implement the UI layer in android application architecture.

Algorithm:

- * start the android application
- * create a new android project in Android studio.

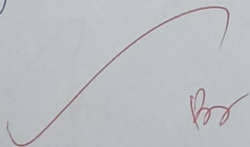
* Import the required packages.

* Run the application in an Android emulator.

* Verify the output.

Result:

The UI layer was implemented successfully.

 For

5:25

LAB 9

Repository Pattern and Work Manager

How it works:

1. Repository manages data from Room DB.
2. Work Manager runs SyncWorker in background.
3. UI updates automatically via Flows.

No goals found. Click 'Sync Now' to start!



+ Sync Now

Aim :

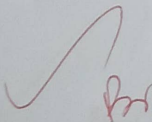
To implement data persistence in an android application.

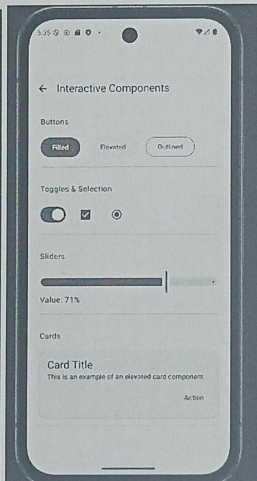
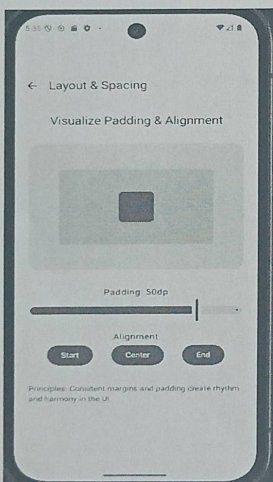
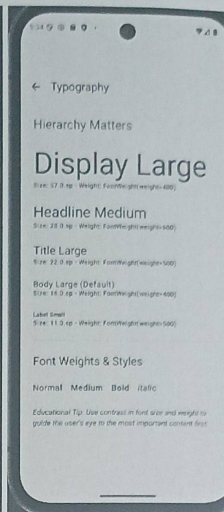
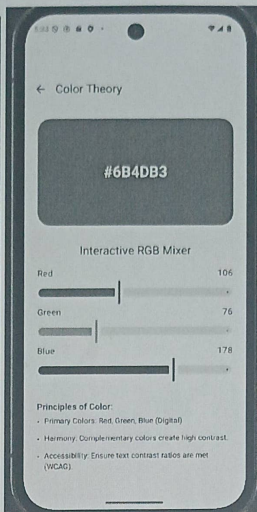
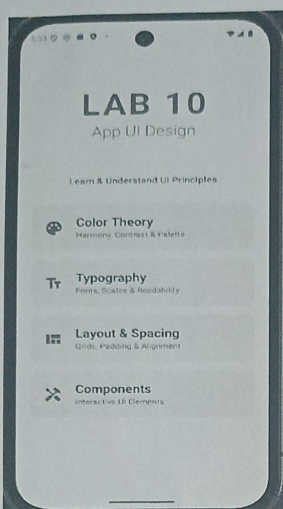
Algorithm :

- * Start the android applications.
- * Create a new Android project in android studio.
- * Import the required packages
- * Run the application in an android emulator.
- * Verify the output.

Result :

Data persistence was implemented successfully.





Advanced RecyclerView View Use Cases

Aim :

To implement advanced use cases of RecyclerView for displaying lists of data.

Algorithm :

- * Start the android application
- * Create a new Android project in Android Studio.
- * Import the required packages.
- * Run the application in an android emulator.
- * Verify the output.

Result :

RecyclerView was implemented successfully, to display dynamic data efficiently.


Pass